

Quotation Generator — Complete Technical Documentation

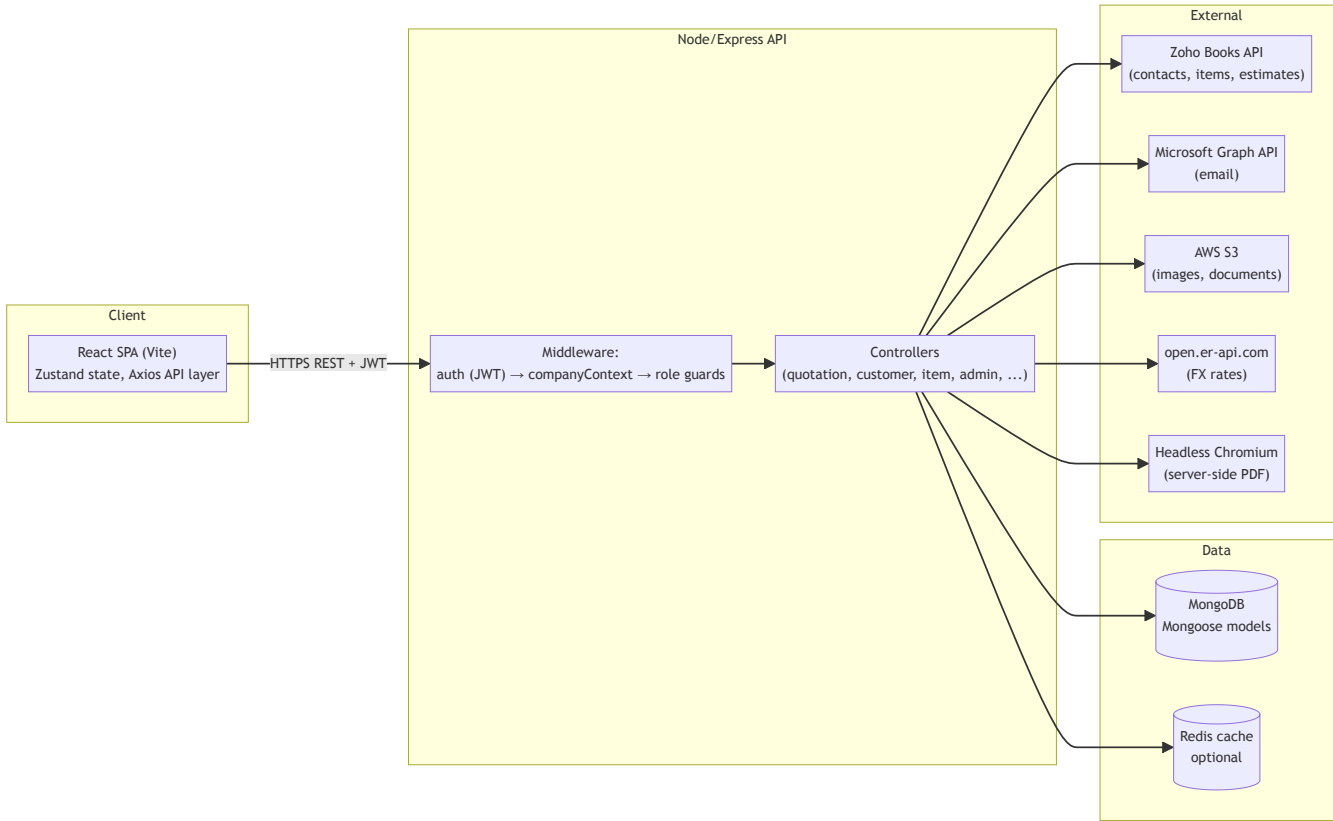
Methodology note: This document was produced by directly reading the source code at `/Users/zcrew/Desktop/Zcrew/quotation-generator-main` (backend + frontend). Every claim below is either a direct code citation or is explicitly flagged as *inferred*. No recommendations, fixes, or optimizations are included — this is a discovery/documentation pass only.

Table of Contents

- 1. [High-Level Overview](#)
 - 2. [Folder Structure](#)
 - 3. [Module Discovery](#)
 - 4. [User Roles](#)
 - 5. [Navigation Map](#)
 - 6. [API Documentation](#)
 - 7. [Database Documentation](#)
 - 8. [Business Workflows](#)
 - 9. [Business Rules](#)
 - 10. [Calculations](#)
 - 11. [State Machines](#)
 - 12. [Authentication & Authorization](#)
 - 13. [Integrations](#)
 - 14. [Configuration](#)
 - 15. [Code Architecture](#)
 - 16. [Component Dependency Map](#)
 - 17. [End-to-End Data Flow](#)
 - 18. [Unknown Areas](#)
-

1. High-Level Overview

Application Name	Quotation Generator (backend <code>package.json</code> name: <code>quotation-system-backend</code>)
Purpose	Multi-tenant B2B sales-quotation authoring, review/approval, and award system, with a two-way sync to Zoho Books (customers, items, and awarded quotations pushed as Zoho estimates).
Business Domain	B2B sales operations / quoting for one or more trading companies (multi-company/multi-tenant), in the UAE/GCC region specifically (VAT treatment, GCC country lists, AED-based currency handling, UAE emirates list all hardcoded as domain constants).
Primary Users	Internal staff of the operating companies: salespeople who create quotations, operations managers who perform a first-pass review, and admins who give final approval and manage the system. There is no external customer-facing login — "Customer" is a CRM record, not a user account.
User Roles	<code>user</code> (quotation creator / sales rep), <code>ops_manager</code> (first-line reviewer), <code>admin</code> (final approver + system administrator). (A fourth value, <code>'customer'</code> , appears in one frontend hook (<code>useUserRole</code>) but is not used by any route guard or backend role check — inferred to be vestigial.)
High-Level Architecture	Classic 3-tier SPA: React SPA (Vite-built, served as static files behind Nginx) ↔ REST API (Node/Express) ↔ MongoDB, with Redis as an optional cache layer and outbound integrations to Zoho Books, Microsoft Graph (email), AWS S3 (file storage), and Puppeteer for server-side PDF rendering.
Technology Stack	See below.
Frontend Framework	React 19 + React Router 7, built with Vite 7, styled with Tailwind CSS 4, state managed with Zustand (no Redux/Context-API global store).
Backend Framework	Node.js + Express 4.
Database	MongoDB via Mongoose 7 (single database, <code>companyId</code> -scoped multi-tenancy at the document level, not separate databases).
Deployment Architecture	Frontend: multi-stage Docker build → static files served by Nginx (<code>nginx.conf</code> does SPA fallback routing only, no API reverse-proxy — the built frontend calls the backend at an absolute URL baked in at build time via <code>VITE_API_URL</code>). Backend: a plain Node process (<code>server.js</code>), with a <code>pm2 ecosystem.config.js</code> present (inferred to be the process manager used in production, since PM2 config exists at the backend root).
Third-Party Services	Zoho Books (customers, items, estimates), Microsoft Graph API (transactional email — not SMTP/nodemailer despite that package being installed), AWS S3 (image/document storage + presigned URLs), Cloudinary (legacy/secondary image storage, still configured and used for some upload paths), Redis (optional response/query caching), open.er-api.com (currency exchange rates), Puppeteer/headless Chromium (server-side PDF rendering).
Authentication Method	JWT bearer tokens, issued on login, stored client-side in <code>localStorage</code> , sent as <code>Authorization: Bearer <token></code> on every request. No refresh-token flow — a single token with an expiry (checked in <code>middleware/auth.js</code>); a 401 anywhere triggers a hard client-side logout-redirect.
Authorization Model	Role-based access control (RBAC) with exactly three roles (<code>user</code> / <code>ops_manager</code> / <code>admin</code>), enforced via Express middleware (<code>adminOnly</code> , <code>opsManagerOrAdmin</code> , <code>opsManagerOnly</code>) plus in-controller status/ownership checks (e.g. "only the quotation's own creator can award it"). Multi-tenancy is layered on top via a <code>companyId</code> / <code>x-company-id</code> header convention (<code>companyContext</code> middleware), with a special <code>'all'</code> sentinel value that only <code>admin</code> users are allowed to select in the UI to view data across every company at once.



2. Folder Structure

Backend (`backend/`)

```
backend/
├─ server.js           - app entry point: middleware wiring, route mounts, DB/Redis connect, cron start
├─ config/             - cross-cutting service configuration (DB connection, logger, Cloudinary, Redis)
├─ controllers/        - request handlers; one file per resource, holds validation + business logic
├─ routes/             - Express routers; map HTTP method+path → controller function + middleware chain
├─ middleware/         - auth (JWT), companyContext (multi-tenant scoping), error handling, upload, HTTP logging
├─ models/             - Mongoose schemas (one file per collection, plus a shared `constants.js`)
├─ zoho/               - Zoho Books API client(s): OAuth token management, contacts/items/estimates calls
├─ cron/               - scheduled jobs (TRN expiry deactivation; an exchange-rate job also exists but is not wired up)
├─ utils/              - stateless/service helpers: email sending, image compression, PDF-adjacent logging, sanitization, sync prog
├─ scripts/            - one-off/maintenance scripts (DB migration, company seeding, Excel import) run manually via `node`/`npm scri
├─ logs/               - Winston output (rotated daily in production)
└─ migrateQuotations.js - a root-level legacy migration script (not wired to package.json)
```

Why each exists:

- **controllers/** holds essentially all business logic in this codebase — there is no separate "service layer" for most domains (Quotation, Customer, Admin, Company, Item). Validation, DB writes, email triggering, and Zoho calls are interleaved directly in controller functions.
- **routes/** is a thin mapping layer only — it wires middleware chains (e.g. `protect` → `companyContext` → `adminOnly`) in front of controller functions; it contains no logic of its own.
- **middleware/** exists to keep cross-cutting request concerns (who is this user, which company are they scoped to, is the request logged) out of every individual controller.
- **models/** centralizes schema definition, indexes, and Mongoose hooks (e.g. the Quotation model's `pre('save')` re-snapshotting logic) so controllers don't have to replicate that logic.
- **zoho/** is isolated because it's the most complex external integration (OAuth token lifecycle, three different resource types, retry/circuit-breaker logic) and is used from multiple controllers (`customerController.js`, `quotationController.js`, `itemController.js`).
- **utils/** is a grab-bag of stateless helpers used by controllers; it's also where a fair amount of **dead code** has accumulated over time (see §18) — several of these files are superseded-but-not-deleted alternate implementations.
- **scripts/** is explicitly for one-time or admin-run operations that aren't part of the live request/response cycle (DB migrations, company seeding).

Frontend (`frontend/quotation/src/`)

```
src/
├─ App.jsx, main.jsx   - router setup + route guards, React entry point
├─ page/               - one file per top-level route/screen (not "pages/")
├─   └─ ViewQuotationScreen/ - sub-components decomposing the largest page
├─ components/         - reusable UI building blocks, shared across pages
├─   └─ DocumentManager/ - internal-document upload/list/preview widget set
├─   └─ HomePageComponent/ - dashboard-specific stat cards/toggles
├─ hooks/              - custom React hooks wrapping store/API access
├─ services/           - API layer (`api.js`) + Zustand stores (`store.js` + 3 domain-specific stores)
├─   └─ store/         - an older, unused slice-based store implementation
├─ utils/              - formatting, PDF generation, validation, image upload, rich-text helpers
└─ assets/             - static images (header logos)
```

Why each exists:

- **page/** is one file per route-level screen; this is where data-fetching hooks are invoked and where role/status-based UI branching happens.
- **components/** exists to share rendering logic between the create-quotation flow (`QuotationTemplate.jsx`) and the view/edit-quotation flow (`ViewQuotationScreen.jsx`) — both render through the same `QuotationLayout` / `MobileQuotationLayout` components rather than duplicating the entire form twice.
- **hooks/** exists to keep pages from directly wiring up fetch/pagination/filter boilerplate — though in practice (see §18) this has resulted in the same concept (e.g. "list of customers") being re-implemented in 2-3 different hook files.
- **services/** is the single place that knows how to talk to the backend (axios instance, interceptors, caching/dedup) and the single place holding client-side global state (Zustand stores) — kept separate from `components` / `page` so UI code doesn't embed HTTP details.
- **utils/** holds pure/stateless helper logic (currency formatting, number-to-words for the PDF, the S3 image URL cache, HTML sanitization) that multiple pages/components need but that isn't itself UI or state.

There is no `contexts/`, `public/` (beyond Vite's default), `Docker` -as-a-folder (a single `Dockerfile` + `nginx.conf` exist at the frontend root instead), or `infrastructure/` folder in this project — deployment config is minimal (see §14).

3. Module Discovery

For each module: purpose, main screens, backend APIs, DB collections, and primary workflow. ("Related Components/Services" and "Dependencies" are folded into the narrative to avoid repeating the API tables in §6.)

3.1 Authentication & User Management

- **Purpose:** Login/registration, session issuance, and (admin-only) full user lifecycle management.
- **Main Screens:** `LoginScreen`, `RegisterScreen`, `UserManagementScreen` (admin only).
- **Backend APIs:** `authRoutes.js` → `authController.js` (see §6).
- **DB Collection:** `User`.
- **User Roles Involved:** all roles log in; only `admin` manages other users.
- **Primary Workflow:** Register/Login → JWT issued → token stored client-side → attached to every subsequent request.
- **Business Rules:** password min length 6, bcrypt-hashed; `isActive` flag can deactivate an account without deleting it; `mustChangePassword` flag forces a password change flow (inferred from field name — the exact enforcement point wasn't traced in this pass).

3.2 Company Management

- **Purpose:** Defines the tenants ("companies") of the system — the unit multi-tenancy is scoped by.
- **Main Screens:** `CompanyManagementScreen` (admin only); every other screen consumes the active company via `CompanyCurrencySelector`.
- **Backend APIs:** `companyRoutes.js` → `companyController.js`.
- **DB Collection:** `Company`.
- **User Roles Involved:** admin manages companies; every role's requests are scoped to a company via the `x-company-id` header.
- **Primary Workflow:** Admin creates a company (code, Zoho org id, base currency, bank details) → the company becomes selectable in the company switcher → every quotation/customer/item created "under" that company stores its `companyId`.
- **Business Rules:** company `code` and `slug` are unique; `zohoOrganizationId` is optional but required specifically to award quotations (Zoho push); only `admin` users can select the special `'all'` company (view across every company).

3.3 Customer Management (CRM)

- **Purpose:** Maintains the customer/client database, each with one or more named contact persons, tax treatment, and a two-way Zoho sync.
- **Main Screens:** `CustomersScreen`.
- **Backend APIs:** `customerRoutes.js` → `customerController.js`.
- **DB Collection:** `Customer` (embeds `contactPersons[]`).
- **User Roles Involved:** any authenticated role (route is not role-restricted, though UI behavior around "view all companies" differs for admins).
- **Primary Workflow:** Create/edit customer (locally + pushed to Zoho synchronously if the company is Zoho-linked) → customer becomes selectable when creating a quotation → contact person chosen/typed on a quotation is synced back onto the customer's `contactPersons[]`.
- **Business Rules:** a customer's own `email` / `phone` are distinct from any `contactPersons[].email/phone`; TRN (`taxRegistrationNumber`) is cleared automatically if the customer isn't VAT-registered; a synchronous Zoho contact-creation call must succeed before a new customer is saved locally, if the company is Zoho-linked.

3.4 Item/Product Catalog

- **Purpose:** A read-only (from this app's perspective) catalog of sellable items/services, sourced from Zoho Books.
- **Main Screens:** `ItemsScreen`.
- **Backend APIs:** `itemRoutes.js` → `itemController.js` (create/update/delete intentionally return HTTP 501 — items are managed in Zoho, not here).
- **DB Collection:** `Item`.
- **User Roles Involved:** any authenticated role.
- **Primary Workflow:** Sync pulls items from Zoho into the local `Item` collection (at server boot and on-demand) → items are selected onto a quotation's line items during quotation creation.
- **Business Rules:** items are company-scoped (unique per `{companyId, zohoId}`); `can_be_sold` / `isActive` / `status` gate whether an item is offerable.

3.5 Quotation Authoring & Lifecycle

- **Purpose:** The core module — create, edit, review, approve, award, cancel, revise, and amend sales quotations.
- **Main Screens:** `QuotationScreen` + `QuotationTemplate` (create), `ViewQuotationScreen` (view/edit/approve/award).
- **Backend APIs:** `quotationRoutes.js` → `quotationController.js` (by far the largest controller).
- **DB Collection:** `Quotation` (embeds items, review comments, internal documents, snapshots of customer/company data at creation time).
- **User Roles Involved:** all three — `user` / `ops_manager` / `admin` create; `ops_manager` performs first review; `admin` gives final approval; the **original creator** (regardless of role) is the only one who can award.
- **Primary Workflow:** see §8.1.
- **Business Rules:** see §9 and the full state machine in §11.

3.6 Review Comments (inline commenting)

- **Purpose:** Lets a reviewer (ops_manager/admin) highlight a specific piece of quotation text (a line item's description/qty/price, a header field, or the Terms & Conditions block) and attach a comment, without a fixed list of "commentable" targets baked into the schema.
- **Main Screens:** embedded inside `ViewQuotationScreen` via `ReviewComments.jsx`'s `CommentableText / CommentableHtml / CommentBadge`.
- **Backend APIs:** `POST/PATCH/DELETE /api/quotations/:id/comments...` (part of `quotationRoutes.js / quotationController.js`, not a separate resource).
- **DB Collection:** embedded sub-documents on `Quotation.reviewComments[]`.
- **User Roles Involved:** only `ops_manager / admin` can add comments; the comment author or admin can edit/delete; the quotation's creator or admin can resolve.
- **Primary Workflow:** Reviewer selects text → composer opens → comment posted, anchored by the selected text + surrounding context (not character offsets, so it degrades gracefully if the underlying text is later edited).

3.7 Payment Terms & Terms-and-Conditions Templates

- **Purpose:** Two small, structurally identical "reusable saved value" features: a free-text Payment Terms field that remembers previously-used phrasings, and a rich-text Terms & Conditions block that can be saved as a named template and reloaded later.
- **Main Screens:** the Payment Terms field and the Terms & Conditions section of `QuotationTemplate / ViewQuotationScreen` (via `PaymentTermPicker.jsx / TermsTemplateControls.jsx`).
- **Backend APIs:** `paymentTermRoutes.js / termsTemplateRoutes.js` → their respective controllers.
- **DB Collections:** `PaymentTerm`, `TermsTemplate`.
- **User Roles Involved:** any authenticated role that can create/edit a quotation.
- **Primary Workflow:** value saved once (or automatically, for payment terms, as a side effect of any quotation save) → reused via a dropdown on future quotations.
- **Business Rules:** both collections are **global, not company-scoped** — a value saved under any company is visible/reusable from every company (the `companyId` field on each is recorded for reference only). Both enforce a case-insensitive uniqueness constraint on their value/name globally.

3.8 Admin & Ops Dashboards / Reporting

- **Purpose:** Role-specific overviews of quotation activity — pending review queues, approval actions, stats, Excel export, per-user performance stats.
- **Main Screens:** `AdminDashboard`, `OpManagerDashboard`, `UserQuotationStatsPage`.
- **Backend APIs:** `adminRoutes.js` → `adminController.js`.
- **DB Collection:** reads `Quotation / User` (no dedicated collection).
- **User Roles Involved:** admin (both dashboards' data), `ops_manager` (own dashboard).
- **Primary Workflow:** dashboard loads aggregate stats + a filtered/paginated quotation list; approve/reject actions are dispatched from here.

3.9 Notifications (in-app)

- **Purpose:** A generic in-app notification model/API intended to surface events (submission, approval, rejection, award, etc.) to users.
- **Main Screens:** `NotificationDrawer.jsx` component exists but — per direct code search — is **not rendered from any page**, i.e. this module has a live, mounted API but no reachable UI.
- **Backend APIs:** `notificationRoutes.js` → `notificationController.js` (fully live and mounted).
- **DB Collection:** `Notification`.
- **Status:** the service classes that would *write* into this collection along the real quotation-review workflow (`utils/notificationService.js`, `utils/approvalWorkflowService.js`) are not invoked from the live approve/reject/award code paths — see §18. This module is architecturally complete but not wired end-to-end in the current codebase.

3.10 Exchange Rates

- **Purpose:** Currency conversion support so a quotation issued in one currency can be reported/compared in the company's base currency.
- **Main Screens:** no dedicated screen; consumed transparently via `CompanyCurrencySelector / LocalCurrencySelector` and quotation total calculations.
- **Backend APIs:** `exchangeRates.js` → `exchangeRateController.js`.
- **DB Collection:** `ExchangeRate` (rates cached with a 1-hour TTL index).
- **Primary Workflow:** rate fetched live from `open.er-api.com` on demand, with a DB-cache fallback, then a hardcoded fallback table if both fail.

3.11 PDF Generation

- **Purpose:** Produces a downloadable/printable PDF rendition of a quotation.
- **Main Screens:** "Download PDF"/"Print" actions on `QuotationTemplate / ViewQuotationScreen`.
- **Backend APIs:** `POST /api/quotations/generate-pdf`.
- **Primary Workflow:** frontend builds a full HTML string client-side → for **Print**, written directly into an iframe and printed via the browser (no server round-trip); for **Download**, POSTed to the backend, rendered by a pooled headless-Chromium (Puppeteer) instance with all non-image network requests blocked, returned as a PDF binary.

3.12 Zoho Integration

- **Purpose:** Keeps customers, items, and awarded quotations in sync with Zoho Books.
 - **Backend APIs:** no direct frontend-facing routes of its own — invoked internally from `customerController.js`, `itemController.js`, and `quotationController.js`.
 - **Primary Workflow:** see [§13.1](#).
-

4. User Roles

	user (creator/sales)	ops_manager	admin
Purpose	Creates and manages their own quotations	First-pass reviewer of submitted quotations	Final approver + full system administrator
Permissions	Create/edit own quotations while in an editable status; add customers/items; cannot approve/reject at any stage	Everything a user can do, plus: ops-approve/ops-reject quotations in pending status; add/manage review comments	Everything, plus: final approve/reject, delete any quotation, manage users/companies, view/act across "All Companies", export Excel reports
Accessible Modules	Home dashboard, Customers, Items, Create/View Quotation	Ops dashboard, Customers, Items, Create/View Quotation, Review Comments	Admin dashboard, User Management, Company Management, User Stats, Customers, Items, Create/View Quotation, Review Comments
Restricted Modules	/admin/* , /ops	/admin/*	none — admin can reach every route
Dashboard	/home — own quotations, status tabs, stats	/ops — pending-review queue + ops review history + own-created-quotation award tracking	/admin — company-wide (or all-companies) quotations, approve/reject queue, Excel export
Allowed Actions	Create quotation, edit while in an editable status, resubmit after rejection, award/mark-not-awarded a quotation they created, cancel/amend their own pre-approval quotation	+ ops-approve/ops-reject a pending quotation, add/edit/delete/resolve review comments, award their own approved quotations	+ admin-approve/admin-reject ops_approved / pending_admin quotations, delete any quotation, cancel an approved quotation, manage users (create/deactivate/change role/reset password), manage companies, select "All Companies"
Hidden Features	Approve/reject actions, admin/ops dashboards, user/company management, "All Companies" selector	Admin-only final-approval action, user/company management, "All Companies" selector	none observed — the UI does not appear to hide any feature from admin

(Company assignment: confirmed the User model has **no companyId field** — an ops_manager is not restricted to a single company at the data layer. Any per-company restriction observed for ops_manager in the frontend/company-switcher is a UI/request-scoping behavior, not a stored permission — this is stated as a direct finding, not an inference, since getOpsManagers in authController.js queries all active ops_managers with no company filter.)

5. Navigation Map

Screen (route)	Purpose	Accessible By	Key Actions	Connected APIs
/login	Sign in	Guests only (redirects away if authenticated)	Submit credentials	authAPI.login
/register	Self-registration	Guests only	Submit new-account form	authAPI.register
/home	Creator's own quotation dashboard	user role only	Filter/search/sort own quotations, create new, view/PDF/delete/award, set follow-up "query date", switch company/currency	quotationAPI.getMyQuotations , .getMyQuotationsStats , dashboard stats
/customers	Customer CRM	any authenticated role	Add/edit/delete customer, filter (tax status, place of supply, Zoho sync status), Zoho sync (with live progress modal), Excel export	customerAPI.*
/items	Item catalog	any authenticated role	Search/filter (product type, sellable/active), sync from Zoho (with progress)	itemAPI.*
/quotation/new	Create a quotation (2-step: select customer/items → author quotation)	any authenticated role	Select customer + items → fill header fields, line items, terms, payment terms, upload documents → submit	customerAPI , itemAPI , quotationAPI.create , termsTemplateAPI , paymentAPI
/quotation/:id	View/edit/act on a single quotation	any authenticated role (with per-status/role action gating)	Edit (if editable status + permitted role), approve/reject/ops-approve/ops-reject, award/not-award, cancel/amend, add review comments, download PDF, manage internal documents	quotationAPI.getById/update/approve/reject/opsApprove/opsReject/award comment endpoints
/ops	Ops manager's review dashboard	ops_manager role only	Ops-approve/ops-reject pending quotations, view ops review history, award own quotations	opsAPI.*
/admin	Admin's company-wide (or all-companies) dashboard	admin role only	Approve/reject, delete, award, Excel export, date-range/status filtering	adminAPI.*
/admin/users	User management	admin role only	Create/edit/deactivate users, change role, reset/generate password	authAPI (user-management endpoints)
/admin/user-stats	Per-user quotation performance	admin role only	Drill into a user's quotations, sort/search, per-quotation PDF export	adminAPI.getUserQuotationStats , .getQuotationsByUser
/admin/companies	Company management	admin role only	Create/edit/deactivate companies	companyAPI.*
/ and unmatched paths	Root redirect	anyone	Redirects to /login (no user) or the role's home path	—

Entry points: the app has a single entry (`main.jsx` → `App.jsx` 's router). Every screen except `/login / /register` requires a valid token + user object in `localStorage` ; role-mismatched access attempts are silently redirected to that user's *own* home path rather than shown a 403 page (confirmed in `App.jsx` 's guard components).

6. API Documentation

All routes are mounted under `/api` (confirmed in `server.js`). `protect` = JWT auth middleware; `companyContext` = multi-tenant scoping middleware; role names refer to the middleware functions in `middleware/auth.js`.

6.1 Auth (`/api/auth`)

Method	Route	Purpose	Auth	Notes
POST	<code>/register</code>	Create a new user account	none	
POST	<code>/login</code>	Issue a JWT	none	
GET	<code>/me</code>	Get current user profile	<code>protect</code>	
PUT	<code>/updatedetails</code>	Update own profile	<code>protect</code>	
PUT	<code>/updatepassword</code>	Change own password	<code>protect</code>	
GET	<code>/users</code>	List all users	<code>protect</code> + admin	
GET	<code>/ops-managers</code>	List active ops managers	<code>protect</code>	no company filter applied
GET/PUT/DELETE	<code>/users/:id</code> / <code>/users/:userId</code>	Get/update/delete a specific user	<code>protect</code> + admin	
PUT	<code>/users/:id/toggle-status</code>	Activate/deactivate a user	<code>protect</code> + admin	
PUT	<code>/users/:id/role</code>	Change a user's role	<code>protect</code> + admin	
POST	<code>/users/:userId/send-reset-password</code>	Trigger password-reset email	<code>protect</code> + admin	
PUT	<code>/users/:userId/set-password</code>	Directly set a user's password	<code>protect</code> + admin	
POST	<code>/users/:userId/generate-temp-password</code>	Generate a temporary password	<code>protect</code> + admin	
PUT	<code>/reset-password</code> , <code>/force-change-password</code>	Self-service password reset flows	varies	

6.2 Quotations (`/api/quotations`)

Method	Route	Purpose	Auth	Called By (UI)
GET	<code>/companies</code> , <code>/companies/:code</code> , <code>/companies/:code/stats</code>	Company lookups scoped for the quotation flow	<code>protect</code>	<code>QuotationScreen</code>
GET	<code>/my-quotations</code> , <code>/my-quotations/stats</code>	Own quotations + stats	<code>protect</code>	<code>HomeScreen</code>
GET	<code>/</code>	All quotations (filterable/paginated)	<code>protect</code> + admin	<code>AdminDashboard</code>
GET	<code>/:id</code>	Single quotation (with visibility rules — see §8.1)	<code>protect</code>	<code>ViewQuotationScreen</code>
POST	<code>/</code>	Create a quotation (also handles revision-of-cancelled)	<code>protect</code>	<code>QuotationTemplate</code>
PUT	<code>/:id</code>	Update a quotation (also handles resubmit/amend/revise-in-place)	<code>protect</code>	<code>ViewQuotationScreen</code> (edit mode)
DELETE	<code>/:id</code>	Delete a quotation (role/status-gated)	<code>protect</code>	<code>ViewQuotationScreen</code>
PATCH	<code>/:id/query-date</code>	Set a follow-up reminder date	<code>protect</code>	<code>HomeScreen</code> / <code>QueryDateUpdater</code>
PATCH	<code>/:id/cancel</code>	Cancel (post-approval) or amend (pre-approval)	<code>protect</code> + role checks in-controller	<code>ViewQuotationScreen</code>
PATCH	<code>/:id/award</code>	Award / mark-not-awarded (creator-only; Zoho push on award)	<code>protect</code>	<code>ViewQuotationScreen</code>
POST	<code>/presign-image</code>	Get an S3 presigned upload URL	<code>protect</code>	item/terms image upload
GET	<code>/signed-url/:s3Key</code> , <code>POST /signed-urls/batch</code>	Get signed read URLs for S3 objects	<code>protect</code>	image/document display
POST	<code>/:id/comments</code>	Add a review comment	<code>protect</code> + ops_manager/admin	<code>ReviewComments</code>
PATCH	<code>/:id/comments/:commentId</code>	Edit a review comment	<code>protect</code> + author/admin	<code>ReviewComments</code>
PATCH	<code>/:id/comments/:commentId/resolve</code>	Resolve a review comment	<code>protect</code> + creator/admin	<code>ReviewComments</code>
DELETE	<code>/:id/comments/:commentId</code>	Delete a review comment	<code>protect</code> + author/admin	<code>ReviewComments</code>
POST	<code>/generate-pdf</code>	Server-side PDF render (Puppeteer)	<code>protect</code>	download-PDF action
GET	<code>/pdf-metrics</code>	Puppeteer pool metrics	<code>protect</code> + admin	ops tooling (not user-facing)

Method	Route	Purpose	Auth	Called By (UI)
Internal-documents sub-routes	<code>/:id/internal-documents...</code>	Upload/list/preview/update/delete/download attached documents	<code>protect</code>	<code>DocumentManager</code>

6.3 Admin (`/api/admin`)

Method	Route	Purpose	Auth
GET	<code>/dashboard</code>	Admin dashboard stats	<code>admin</code>
GET	<code>/quotations</code> , <code>/quotations/pending</code>	Admin's quotation list views	<code>admin</code>
PUT	<code>/quotations/:id/approve</code> , <code>/quotations/:id/reject</code>	Final approve/reject	<code>admin</code>
GET	<code>/quotations/ops-pending</code> , <code>/quotations/ops-history</code> , <code>/quotations/ops-all</code>	Ops manager's queues	<code>ops_manager</code> or <code>admin</code>
PUT	<code>/quotations/:id/ops-approve</code> , <code>/quotations/:id/ops-reject</code>	First-pass review decision	<code>ops_manager</code> or <code>admin</code>
GET	<code>/ops-dashboard</code>	Ops dashboard stats	<code>ops_manager</code> or <code>admin</code>
GET	<code>/user-stats</code> , <code>/user-quotations/:userId</code>	Per-user performance stats	<code>admin</code>
GET	<code>/export-excel</code>	Excel export of quotations	<code>admin</code>

6.4 Customers (`/api/customers`)

CRUD (`GET/POST/PUT/DELETE`), search, Zoho sync (`sync-from-zoho` + `progress/cancel/status/pending` endpoints, plus a per-customer `force-sync`), stats, GCC country/currency/tax-treatment reference lists, Excel export, tax-summary, bulk create, place-of-supply stats. All behind `protect` + `companyContext` .

6.5 Items (`/api/items`)

`GET /` , `GET /:id` , `POST /sync` (+ `progress/status`). `POST/PUT/DELETE` are routed but return HTTP 501 by design (items are Zoho-managed).

6.6 Companies (`/api/companies`)

CRUD + `GET /:id/stats` , `GET /:id/currencies` , `GET /code/:code` , `PATCH /:id/toggle-status` , `POST /bulk` . Create/update/delete are admin-only (inferred from `CompanyManagementScreen` being an admin-only route; not independently re-verified per-route in this pass).

6.7 Terms Templates (`/api/terms-templates`) & Payment Terms (`/api/payment-terms`)

Both: `GET /` (list, global — not company-filtered), `POST /` (create/find-or-create), `DELETE /:id` . Behind `protect` + `companyContext` (the header is still read for informational `companyId` recording, but not used to scope results).

6.8 Exchange Rates (`/api/exchange-rates`)

`GET /rates` , `POST /convert` , `GET /history` , `GET /supported` , `POST /refresh` (**note:** confirmed to call a non-existent `ExchangeRateService.forceRefresh` method — see §18), `GET /status` .

6.9 Notifications (`/api/notifications`)

`GET /` , `GET /unread-count` , `GET /:id` , `PUT /:id/read` , `PUT /read-all` , `PUT /:id/archive` , `DELETE /:id` . Live and mounted, but no frontend screen currently renders this data (see §18).

7. Database Documentation

MongoDB via Mongoose. Multi-tenancy is achieved by a required (or, for `PaymentTerm` / `TermsTemplate` , optional/informational) `companyId` field on most collections, not by separate databases.

7.1 User

Field	Type	Notes
name, email, phone, designation	String	email unique+lowercased
password	String	bcrypt-hashed, <code>select:false</code>
role	String enum	<code>user</code> / <code>admin</code> / <code>ops_manager</code> , default <code>user</code>
isActive	Boolean	default true
lastLogin, passwordResetToken/Expires, mustChangePassword, lastPasswordChange	—	account-lifecycle fields

No `companyId` — a global-scope collection. **Referenced by** nearly every other collection (`createdBy` , `approvedBy` , etc.) but has no outgoing `ref` fields itself.

7.2 Company

Field	Type	Notes
code, slug	String	both unique
name	String	required
zohoOrganizationId	String	unique, sparse — "required only to award quotations"
address	<code>{street, city, country, poBox}</code>	embedded object
phone, email, website, logo, vatNumber, crNumber	String	—
taxRate	Number	default 5
baseCurrency	String enum	default AED
acceptedCurrencies	[String enum]	—
bankDetails	<code>{bankName, accountName, accountNumber, iban, swift}</code>	—
isActive	Boolean	default true

Referenced by: `Customer.companyId` , `Item.companyId` , `Quotation.companyId` , `Notification.companyId` , `PaymentTerm.companyId` , `TermsTemplate.companyId` .

7.3 Customer

Field	Type	Notes
companyId	ObjectId ref Company	required
name	String	required, 2–100 chars
zohold	String	unique per <code>{companyId, zohoId}</code> (partial index, string-typed only)
email, phone, address, city, state, zipcode, companyName, website, notes	String	customer's own contact info
taxTreatment	String enum	required, default <code>non_vat_registered</code>
taxRegistrationNumber	String	digits-only, cleared if not VAT-registered
contactPersons[]	subdoc array	<code>salutation, firstName, lastName, email, workPhone, mobile, designation, department, isPrimaryContact, zohoContactPersonId</code> — structurally distinct from the customer's own email / phone
primaryContactId	ObjectId	plain, no <code>ref</code>
placeOfSupply	String	required, default <code>Dubai</code>
defaultCurrency	<code>{code, symbol, name}</code>	default AED
zohoSynced, zohoSyncDate, zohoSyncError	—	sync bookkeeping

Field	Type	Notes
trnExpiryDate, tradeLicenseNumber, tradeLicenseExpiryDate	—	compliance fields
isActive	Boolean	default true
createdBy	ObjectId ref User	—

Business meaning: the single source of truth for "who is this quotation being issued to," including which named individual (contact person) is the point of contact — deliberately separate from the company-level email/phone to prevent one from silently overwriting the other during Zoho sync (documented in code comments).

7.4 Item

Field	Type	Notes
companyId	ObjectId ref Company	required
zohold	String	required; unique per {companyId, zohoId}
name, description, sku, unit	String	—
price	Number	required, min 0
product_type	String	default goods
tax_percentage	Number	default 0
status	String	default active (freeform, mirrors Zoho's own status string)
is_taxable, can_be_sold, isActive	Boolean	all default true
lastSyncedAt, lastModifiedTime, zohoData	—	sync bookkeeping; zohoData retains the full raw Zoho payload

Business meaning: a read-only local cache of Zoho Books' item/service catalog, used to populate quotation line items.

7.5 Quotation (the central collection)

Full field-by-field detail is extensive — summarized here by group; see §11 for the status field specifically.

- **Identity/scope:** companyId, customerId, createdBy, quotationNumber (unique per company), projectName.
- **Snapshots (point-in-time copies, not live joins):** companySnapshot (name/address/contact/VAT/CR/Zoho org id/bank details), customerSnapshot (name/contactPerson/email/phone/address/country/VAT/designation/trade license/tax treatment/place of supply). These exist so a quotation's printed content doesn't silently change if the Company/Customer record is edited later.
- **Commercial fields:** currency ({code, symbol, name, decimalPlaces, exchangeRate}), items[] (see below), taxPercent, discountPercent, subtotal / taxAmount / discountAmount / total (+ their ...InBaseCurrency equivalents).
- **Content fields:** scopeOfWork, notes, remark, termsAndConditions (sanitized HTML), termsTemplateId (ref TermsTemplate), termsImages[], internalDocuments[] (see below), paymentTerms, deliveryTerms.
- **Company-reference fields:** ourRef, ourContact, ourFocalPointDesignation, salesManagerEmail, tl, trn.
- **Workflow/status:** status (see §11), full audit trail for each transition (opsApprovedBy/At, opsRejectedBy/At/Reason, approvedBy/At, rejectedBy/At/Reason, awardedBy/At/Note, cancelledBy/At/Reason/FromStatus), revisedFrom / revisionNote / revisionNumber / isRevision, isAmendment / amendmentNote, duplicatedFrom (declared but never populated — see §18).
- **Zoho:** zohoEstimateId / Number / Url, zohoSyncedAt.
- **reviewComments[]** (embedded, see §3.6): targetType (free string, not enum), targetKey, quote / prefix / suffix (anchor), comment, createdBy / createdBySnapshot, resolved / resolvedBy / resolvedAt.

quotationItemSchema (embedded): zohoItemId, description, unit, quantity (min 0.001), unitPrice / totalPrice (+ base-currency equivalents), image fields (imageS3Keys[], legacy imagePath[] / imagePublicIds[]). Each item gets a real _id specifically so review comments can anchor to a stable item.

quotationDocumentSchema (embedded, internal documents): fileName, fileType, fileSize, s3Key / legacy imageUrl / publicId, uploadedAt, uploadedBy (ref User), description.

Indexes: compound indexes on {companyId, status, createdAt}, {companyId, date}, {companyId, customerId}, {companyId, createdBy}, unique {companyId, quotationNumber}, {companyId, totalInBaseCurrency}, {companyId, queryDate}, plus a weighted **text index** (quotationNumber / customerSnapshot.name / contact / projectName) powering free-text search.

Referenced by: Notification.quotationId; self-referential via revisedFrom / duplicatedFrom.

7.6 PaymentTerm / TermsTemplate

	PaymentTerm	TermsTemplate
Unique value field	value (String, max 200)	name (String, max 100) + content (sanitized HTML)
Uniqueness	case-insensitive, global (not per-company)	case-insensitive, global

	PaymentTerm	TermsTemplate
companyId	recorded, not used to scope reads	recorded, not used to scope reads
createdBy	ref User	ref User (+ createdByName snapshot)

7.7 Notification

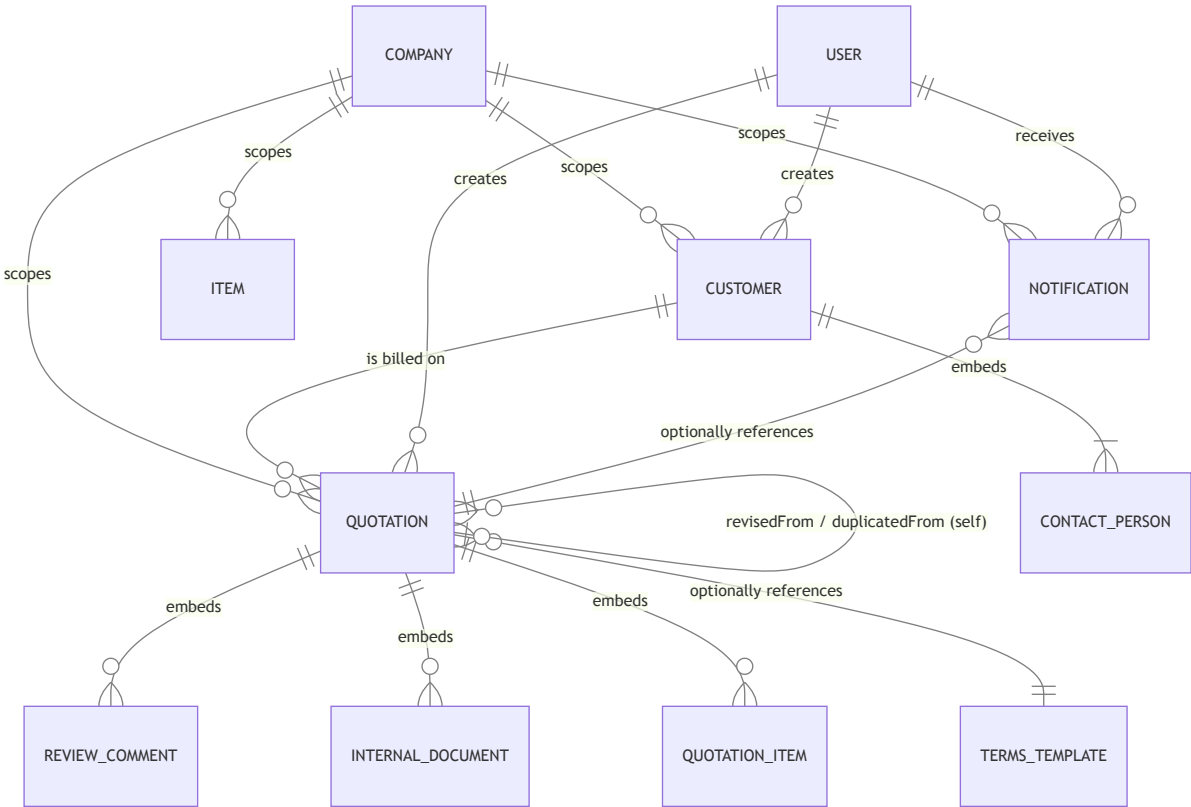
Field	Type	Notes
companyId, userId	ObjectId refs	required
type	enum (11 values)	e.g. quotation_submitted , quotation_approved , revision_requested
title, message	String	required
quotationId, customerId	ObjectId refs	optional
isRead, isArchived	Boolean	default false
priority	enum	low/medium/high/urgent
expiresAt	Date	default now+30 days, TTL index (auto-deletes)

Live schema/API, but no current producer in the real approval workflow (see §18).

7.8 ExchangeRate (defined inside models/quotation.js)

baseCurrency (String, default AED), rates (Map<String,Number>), fetchedAt (Date, **TTL index**, **expireAfterSeconds: 3600** — documents self-expire after 1 hour, acting as a rolling cache).

7.9 Relationships Diagram



8. Business Workflows

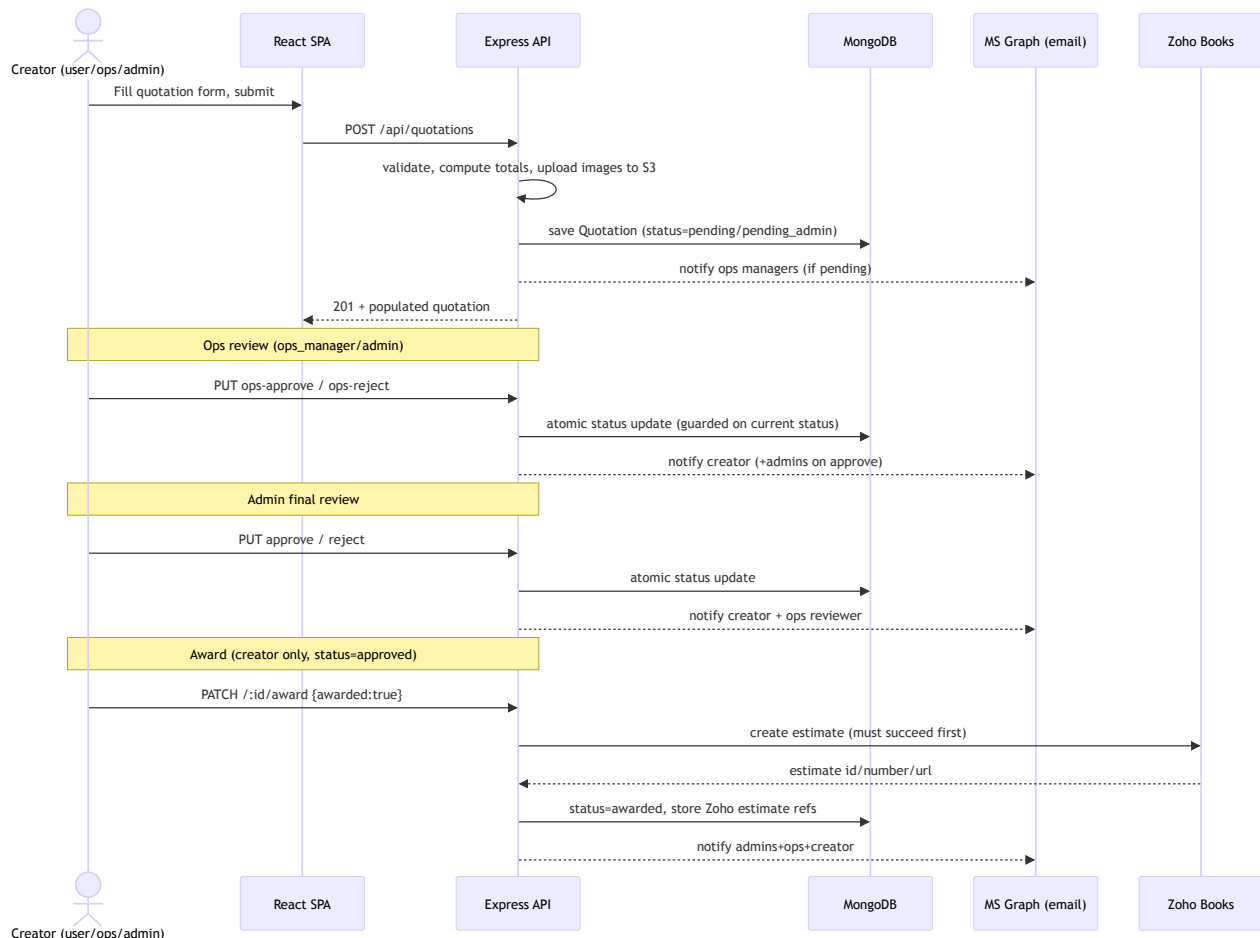
8.1 Quotation Lifecycle (create → review → approve → award)

Trigger: a `user / ops_manager / admin` fills out and submits the create-quotation form.

Steps:

1. Frontend generates a client-side `quotationNumber` (`{companyCode}-{epoch-ms}-{3-digit random}` — see §9) and posts to `POST /api/quotations`.
2. Backend validates required fields (project name, valid company/customer, at least one item with `description/quantity>0`/non-negative price, parseable expiry date), compresses+uploads any embedded images to S3, resolves the exchange rate if the quotation currency differs from the company base currency, computes totals (§10), and sets the initial status: `pending` for `user / ops_manager` creators, `pending_admin` for `admin` creators (admins skip ops review entirely).
3. On successful save: the quotation's contact-person data is synced onto the customer record (locally awaited; the Zoho push of that same contact-person update is fire-and-forget), and — only if the new status is `pending` — an email is sent to ops managers (`emailService.creatorSubmittedNotifyOps`).
4. **Ops review** (only quotations in `pending`): an `ops_manager` (or `admin`) approves (→ `ops_approved` , emails the creator + all other admins) or rejects with a required reason (→ `ops_rejected` , emails the creator).
5. **Admin review** (quotations in `ops_approved` or `pending_admin`): `admin` approves (→ `approved` , emails the creator + the reviewing ops manager) or rejects with a required reason (→ `rejected` , emails the same two parties).
6. **Resubmission:** the creator (or, for `ops_rejected` , an `ops_manager`) can edit an `ops_rejected / rejected` quotation and resubmit — it goes back to `pending` (or straight to `ops_approved` if an `ops_manager` is the one editing/re-approving in the same call) and re-triggers the ops-notification email.
7. **Award:** once `approved` , only the **original creator** can mark it `awarded` (triggers a synchronous Zoho estimate-creation call that must succeed before the DB is touched) or `not_awarded` (no Zoho call). Either way, all admins+ops_managers+the creator are emailed.
8. **Cancel/Amend:** cancelling an `approved` quotation (`admin` only) sets it to `cancelled` and opens up a "revise" path (a brand-new quotation, `revisedFrom` -linked, with an auto-suffixed `-R{n}` number, restarting at `pending / pending_admin`); cancelling anything earlier (`admin` or `ops_manager`) sets it to `amended` and allows an in-place edit that restarts it at `pending` with `isAmendment: true` .

End result: a quotation ends in one of the terminal states `awarded` , `not_awarded` , or (via the amend/cancel-without-approval path) is superseded by a new in-place edit; a `cancelled` (post-approval) quotation is permanently archived once a revision exists for it.



8.2 Customer Creation with Zoho Sync

Trigger: a user submits the "Add Customer" form.

Steps: validate name/TRN/tax data → build `contactPersons[]` (synthesizing an implicit primary contact from the customer's own name/email/phone if none was explicitly provided, to prevent Zoho-sync identity confusion) → if the company has a `zohoOrganizationId`, **synchronously** call Zoho's contact-create API first — a Zoho failure aborts the entire request (nothing is saved locally) → on success, save the `Customer` document locally.

8.3 Bulk Zoho Customer Pull-Sync

Trigger: "Sync from Zoho" button on `CustomersScreen`.

Steps: request returns `202` immediately; sync runs after the response (fire-and-forget), matching/deduping by Zoho's own `zohoId` (not by email), with progress tracked in-memory and pollable/cancellable by the frontend via a dedicated progress endpoint.

8.4 Revision, Amendment, and Duplicate

See the dedicated breakdown in §9 (Revision Rules) — summarized: **Revision** = new document, same customer/company, linked via `revisedFrom`, triggered only from a cancelled-*after*-approval quotation. **Amendment** = same document, in-place edit, triggered from cancelling a *not-yet-approved* quotation. **Duplicate** = intended to be a new, fully independent document (no server-side link persisted — see §18).

8.5 PDF Generation

Trigger: "Download PDF" or "Print" action.

Steps (Print): client builds an HTML string from the quotation data → written into an iframe via `document.write` → `window.print()` — no server involvement. **Steps (Download):** the same HTML string is POSTed to `/api/quotations/generate-pdf` → a pooled headless-Chromium instance (capped concurrency with a queue) loads it with all non-image requests blocked → renders to A4 PDF → streamed back to the client.

8.6 Review Comments

Trigger: an `ops_manager/admin` selects text on a quotation while viewing it.

Steps: frontend captures the selected text + surrounding context (not a character offset, so the anchor survives later edits) → `POST /:id/comments` → comment stored as a `Quotation.reviewComments[]` subdocument → later rendered by re-locating the quote text against the current content, gracefully rendering "unanchored" if the text no longer matches.

9. Business Rules

- **Quotation numbering:** generated **client-side**, format `{companyId}-{epoch-millisecond timestamp}-{3-digit zero-padded random}`; no server-side sequence/counter exists; uniqueness is enforced only by the DB's `{companyId, quotationNumber}` unique index (i.e. a collision is possible in principle but astronomically unlikely given the millisecond+random composition).
 - **Revision numbering:** any existing `-R\d+` suffix is stripped from the root number, existing sibling revisions for that company are counted, and `-R{count+1}` is appended.
 - **Discount rules:** a single discount percentage applies at the whole-estimate level (`discount_type: 'entity_level'`) when pushed to Zoho, applied **before** tax (`is_discount_before_tax: true`) — uniformly for VAT-registered and non-VAT-registered customers alike (the codebase's own comments note an earlier design that baked discount into line items for VAT customers was found to be unnecessary; the real fix for a prior Zoho rejection was correcting `is_discount_before_tax` and the discount value's string format, not changing discount placement).
 - **Tax rules:** tax is computed on the subtotal **after** discount is applied, both frontend and backend (`taxAmount = (subtotal - discountAmount) * taxPercent/100`). Zoho tax IDs are looked up from a hardcoded map keyed by the company's `zohoOrganizationId`; one specific org (Saudi, `932531766`) offers a 15% rate and no 5%, every other configured org offers 0%/5% only; awarding with an unconfigured rate is blocked with a 422 before any Zoho call is made.
 - **Currency rounding:** KWD/BHD/OMR are formatted/displayed to 3 decimal places, every other supported currency to 2 — this is a **display/formatting-layer** distinction; the underlying stored numeric values in MongoDB are always rounded to 3-decimal precision regardless of currency.
 - **Status changes / approval rules:** see the full transition table in §11 — key rules: only `admin` can give final approval/rejection; only `ops_manager` / `admin` can perform the first-pass review; a rejected-by-ops quotation can only be edited back by its own creator or an admin (not by an arbitrary `ops_manager`); admin-created quotations skip ops review entirely; approve/reject transitions use an atomic "guarded on current status" update specifically to prevent a race where two reviewers act on the same quotation simultaneously (returns 409 if the status already changed).
 - **Award rules:** only the quotation's original creator (by user id, not role) can award or mark-not-awarded; award is only possible from `approved`; a successful Zoho estimate push is a hard prerequisite for the status to become `awarded` (failure leaves it `approved`, untouched).
 - **Cancellation rules:** `awarded` / `not_awarded` /already- `cancelled` / `amended` quotations cannot be cancelled again; cancelling an `approved` quotation requires `admin`; cancelling anything earlier permits `admin` or `ops_manager`.
 - **Branch/company restrictions:** only `admin` users can select "All Companies" in the company switcher (`companyContext` middleware's `'all'` sentinel is available to any request that sends it, but the frontend only ever offers the option to admins); `ops_manager` is **not** stored as company-restricted at the data layer (see §4 note).
 - **Permissions (delete):** admin can delete any quotation; a non-admin creator can only delete their own quotation while it's `pending` or `ops_rejected`.
 - **Global (not company-scoped) reference data:** Payment Terms and Terms & Conditions templates are shared across every company once created under any one of them.
 - **Customer TRN:** cleared automatically if the customer's tax treatment isn't a VAT-registered variant.
 - **Price calculations:** see §10 in full.
-

10. Calculations

10.1 Quotation Totals

Formula (identical order on both frontend and backend — discount computed before tax):

```
lineTotal      = quantity × unitPrice                (per item)
subtotal       = ∑ lineTotal
discountAmount = subtotal × discountPercent / 100
subtotalAfterDiscount = subtotal − discountAmount
taxAmount      = subtotalAfterDiscount × taxPercent / 100
grandTotal     = subtotalAfterDiscount + taxAmount
```

- **Inputs:** each line item's `quantity / unitPrice`; the quotation's `discountPercent / taxPercent`.
- **Outputs:** `subtotal`, `discountAmount`, `taxAmount`, `total / grandTotal` (+ each of those again multiplied by the resolved exchange rate for the `...InBaseCurrency` variants).
- **Where used:** `createQuotation / updateQuotation` (backend, `quotationController.js`), and mirrored client-side in `useQuotation.js` for live UI display before save.
- **Edge cases:** currency-specific decimal rounding is applied at *display* time (2 vs. 3 decimals per currency, §9), while every value is stored server-side rounded to a flat 3 decimals regardless of currency.

10.2 Currency Conversion

- **Formula:** `amountInBaseCurrency = amount × exchangeRate.rate`, where `exchangeRate` is resolved once at quotation-creation/update time from the live `ExchangeRateService` (API call → DB-cache fallback → hardcoded fallback table) and stored on the document (not re-derived on every read).
- **Where used:** every monetary field on a `Quotation` has a `...InBaseCurrency` counterpart computed this way.

10.3 Quotation Number Generation

- **Formula:** ``${companyCode}-${Date.now()}-${String(Math.floor(Math.random()*1000)).padStart(3,'0')}``
- **Inputs:** the selected company's `code`; wall-clock time at form-mount; a random 0-999 value.
- **Where used:** client-side, once per new-quotation form mount (`QuotationTemplate.jsx`).
- **Edge cases:** a byte-identical generator function also exists server-side but is dead code (never invoked) — see §18.

10.4 Revision Suffix

- **Formula:** strip any existing `-R\d+$` from the root quotation number, count existing documents (same company) matching `^{root}-R\d+$`, append `-R{count+1}`.
- **Where used:** both the "new document" revision path (`createQuotation`) and the in-place revision path (`updateQuotation`).

10.5 Zoho Estimate Field Mapping

- Not a calculation in the arithmetic sense, but a deterministic transform: quotation fields → Zoho estimate payload fields (`customerId` → `customer_id`, `quotationNumber` → `reference_number`, `discountPercent` → `discount` formatted as a percentage string, `termsAndConditions` → `terms` after HTML-cleaning/truncation, line items requiring a pre-resolved `zohoItemId` per item).
-

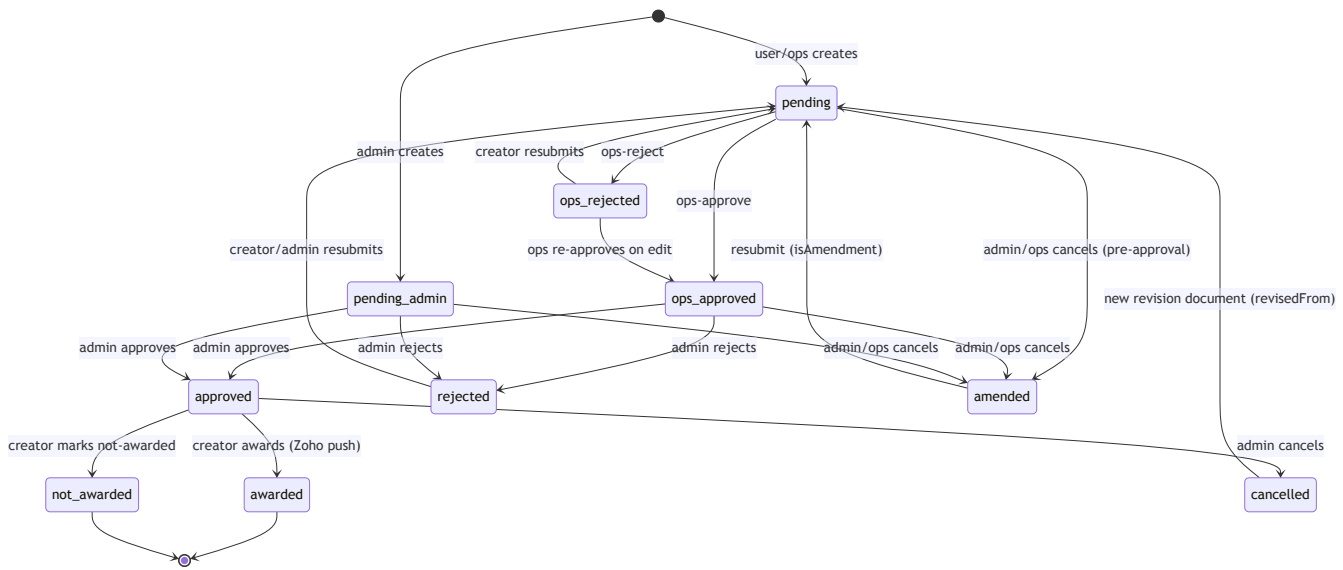
11. State Machines

11.1 Quotation status

Possible statuses (from `models/constants.js`): `draft`, `pending`, `pending_admin`, `ops_approved`, `ops_rejected`, `approved`, `rejected`, `awarded`, `not_awarded`, `sent`, `cancelled`, `amended`.

Note: `draft` and `sent` are declared in the enum but never assigned anywhere in the live controller code (confirmed by direct search) — they exist only in stat-counting arrays and an `editableStatuses` list. They are effectively unreachable states in the current system.

From	To	Trigger	Who
(new)	pending	Create quotation	user / ops_manager
(new)	pending_admin	Create quotation	admin
pending	ops_approved	Ops-approve	ops_manager or admin
pending	ops_rejected	Ops-reject (reason required)	ops_manager or admin
ops_approved pending_admin	approved	Admin-approve	admin
ops_approved pending_admin	rejected	Admin-reject (reason required)	admin
ops_rejected pending	pending or ops_approved	Edit/resubmit	creator (→pending) or ops_manager (→ops_approved)
rejected	pending	Edit/resubmit	creator or admin only
pending pending_admin ops_approved ops_rejected rejected draft	pending_admin	Admin edits it	admin
approved	awarded	Award (Zoho push must succeed)	original creator only
approved	not_awarded	Mark not-awarded	original creator only
approved	cancelled	Cancel	admin only
pending pending_admin ops_approved ops_rejected rejected draft	amended	Cancel (pre-approval)	admin or ops_manager
cancelled	(new document, revision)	Create with revisedFrom	whoever is permitted to create
amended	pending (in-place, isAmendment:true)	Edit/resubmit	creator/ops/admin
awarded not_awarded	(terminal — no further transition)	—	—
cancelled amended	(blocked from cancelling again)	Cancel attempted again	rejected by the API



11.2 Review Comment resolved

Two states only: unresolved (default) → resolved (one-way; no "un-resolve" action found in the controller). Resolve is permitted for the quotation's creator or an admin.

11.3 Customer `isActive`

Two states: active (default) → inactive. Transitioned automatically by the daily TRN-expiry cron job (Zoho deactivation must succeed before the local flag flips) or manually via customer edit.

12. Authentication & Authorization

Login flow: `LoginScreen` → `POST /api/auth/login` → backend verifies email + bcrypt-compares password (rejecting inactive accounts) → signs a JWT → frontend stores `token` + a `user` object (name/email/phone/role/companyId) in `localStorage`.

Token/session flow: stateless JWT, no refresh-token mechanism found — a single token is used until it expires or a 401 forces logout.

`middleware/auth.js`'s `protect` function verifies the token signature/expiry, loads the user (rejecting if `isActive` is false), and attaches `req.user`.

Middleware chain (typical protected route): `protect` (JWT) → `companyContext` (resolves `req.companyId` / `req.isAllCompanies` / `req.company` from the `x-company-id` header, query, or body, special-casing the literal `'all'`) → role guard (`adminOnly` / `opsManagerOrAdmin` / `opsManagerOnly`) if the route needs one → controller.

Protected APIs: effectively every route except `POST /auth/register` and `POST /auth/login`.

Permission checks: role-based via the three middleware functions above, layered with in-controller checks for ownership (e.g. "only the creator can award") and current-status guards (e.g. "can only ops-approve a `pending` quotation").

Role checks: exactly three roles compared via strict string equality (`req.user.role === 'admin'` , etc.) — no permission-bitmask or fine-grained-permission system.

Token lifecycle: issued on login; carries user identity/role (exact claim shape not independently re-verified in this documentation pass beyond "JWT signed with `jsonwebtoken`"); no visible revocation list — deactivating a user (`isActive:false`) is the only server-side way to invalidate their access before natural token expiry, since `protect` re-checks `isActive` on every request.

Password handling: bcrypt hash (`bcryptjs`), min length 6, hashed only when the `password` path is modified (`pre('save')` hook), excluded from query results by default (`select:false`), with dedicated admin-triggered reset/generate-temp-password/force-change-password flows.

13. Integrations

13.1 Zoho Books

- **Purpose:** two-way customer sync, one-way (pull) item catalog sync, one-way (push) quotation→estimate sync at award time.
- **Auth:** OAuth2 refresh-token grant against `accounts.zoho.com` ; access token cached in-memory and in an on-disk file (`.zoho-token.json`), refreshed proactively (5-minute buffer) and reactively (on 401/403), with in-flight-refresh deduplication.
- **Customer sync triggers:** (1) synchronous, blocking push on local customer creation; (2) manual per-customer sync button; (3) manual bulk pull-sync ("Sync from Zoho") which is fire-and-forget with pollable progress.
- **Item sync:** pulled from Zoho at server boot and on-demand; local `Item` records are otherwise read-only (create/update/delete return 501).
- **Quotation→estimate push:** only on "Award," synchronous, must succeed before the local status flips to `awarded` ; tax-rate resolution uses a hardcoded map keyed by the company's Zoho org id; a circuit breaker opens after 5 consecutive Zoho failures (60s cooldown) and errors are classified (`auth` / `rate_limit` / `server_error` / `timeout` / `network` / `client_error` / `circuit_open`) into user-facing messages.

13.2 Microsoft Graph API (Email)

- **Purpose:** all transactional email (submission, ops-approve/reject, admin-approve/reject, award/not-award notifications).
- **Mechanism:** OAuth2 client-credentials grant, then `POST /v1.0/users/{sender}/sendMail` . **Not** SMTP/nodemailer, despite `nodemailer` being present in `package.json` (unused).
- **Fire-and-forget:** email dispatch never throws back to the caller; failures are only logged.

13.3 AWS S3

- **Purpose:** primary storage for quotation item images, Terms & Conditions images, and internal documents.
- **Mechanism:** direct upload from the backend (base64/buffer → S3) plus presigned URLs for client-side reads; batch signed-URL endpoint exists for efficient bulk display.

13.4 Cloudinary

- **Purpose:** legacy/secondary image storage — the `storageProvider` field on image sub-schemas explicitly supports both `'s3'` (current default) and `'cloudinary'` (legacy) values, and a live `utils/uploadCloudinary.js` helper + `config/cloudinary.js` configuration exist and are used for some upload paths (not independently confirmed which specific flows still route through Cloudinary vs. S3 in this documentation pass — flagged as needing a targeted read of upload call sites for a complete answer).

13.5 Redis

- **Purpose:** optional response/query caching (e.g. dashboard stats, "my quotations stats," 5-minute TTL).
- **Behavior:** connection is attempted at boot but is non-blocking — the app continues to function without Redis if the connection fails.

13.6 open.er-api.com

- **Purpose:** live currency exchange rates.
- **Mechanism:** direct HTTPS fetch, DB-cached (1-hour TTL collection), with a final hardcoded fallback table if both the live call and the DB cache are unavailable.

13.7 Puppeteer / Headless Chromium

- **Purpose:** server-side PDF rendering for the "Download PDF" action.
- **Mechanism:** a pooled/queued headless-browser instance (concurrency-capped) renders client-supplied HTML to a PDF buffer.

13.8 Logging / Monitoring

- **Winston** (`config/logger.js`) is the live app-wide logger — daily-rotated file transports in production (error/combined/http logs), console transport always. No external log-shipping/APM/monitoring service (e.g. Datadog, Sentry) was found wired into the codebase.
-

14. Configuration

14.1 Environment Variables (names only, grouped by purpose — confirmed via `process.env`. `grep`)

Group	Variables (representative, not exhaustive)
Database	MONGODB_URI
Auth	JWT secret (referenced in <code>middleware/auth.js</code> / <code>authController.js</code> ; exact variable name not independently re-verified in this pass)
AWS S3	AWS credentials/bucket/region variables consumed by the S3 client setup
Cloudinary	cloud_name / api_key / api_secret -equivalent env vars in <code>config/cloudinary.js</code>
Zoho	ZOHO_CLIENT_ID , ZOHO_CLIENT_SECRET , ZOHO_REFRESH_TOKEN
Email (MS Graph)	AZURE_TENANT_ID , AZURE_CLIENT_ID , AZURE_CLIENT_SECRET , GRAPH_SENDER_EMAIL , SMTP_FROM_NAME , FRONTEND_URL
PDF/Puppeteer	CHROMIUM_PATH , PDF_MAX_CONCURRENT , PDF_MAX_QUEUE
Redis	Redis connection variables consumed by <code>config/redisService.js</code>
Frontend build	VITE_API_URL , VITE_API_BASE_URL (the latter appears unreferenced by the actual app code — see §18)

14.2 Configuration Files

- Backend: `config/db.js` (Mongo connection, with a **hardcoded fallback Atlas connection string containing plaintext credentials** if `MONGODB_URI` is unset — a direct code finding, not a judgment call), `config/logger.js` , `config/cloudinary.js` , `config/redisService.js` .
- Frontend: `vite.config.js` (React + Tailwind v4 Vite plugins, dev server on `0.0.0.0:3000`), `tailwind.config.js` / `postcss.config.js` .

14.3 Docker / Build / Deployment

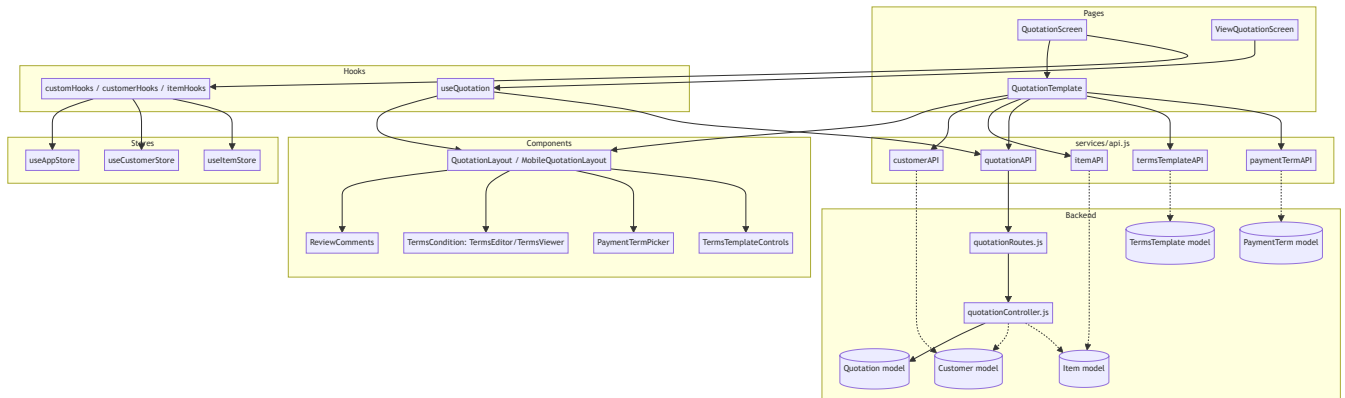
- Frontend:** `Dockerfile` (two-stage: `node:20` build → `nginx:alpine` serve), `nginx.conf` (SPA fallback routing only, no API proxy), `.dockerignore` .
- Backend:** `Dockerfile` presence and content were confirmed to exist by one research pass (a `pm2 ecosystem.config.js` was found at the backend root, implying PM2 process management in the target deployment) — full `Dockerfile` contents were not independently re-verified line-by-line in the final synthesis pass; flagged as an area for a direct read if exact build steps are needed.
- Build commands:** frontend `npm run dev` (Vite dev server) / `npm run build` (Vite build → `dist/`); backend started via `node server.js` (or via PM2 in production, per the ecosystem config's presence).
- Runtime dependencies:** see the tech-stack table in §1 and the full dependency tables gathered per-side; no separate lockfile-drift issues were investigated in this pass.

15. Code Architecture

- **Application architecture:** a fairly conventional **MVC-ish REST API** on the backend (Model = Mongoose schemas, Controller = the request-handling+business-logic files, no separate "View" since responses are JSON) paired with a **component-based SPA** on the frontend. There is **no dedicated service layer** on the backend — controllers call models, external SDKs (Zoho/S3/email), and utility functions directly; this is a structural characteristic of the codebase, not a judgment.
 - **Design patterns observed:**
 - **Snapshot pattern** — `Quotation.companySnapshot / customerSnapshot` freeze company/customer data at creation time so a printed quotation doesn't retroactively change if the source record is edited later.
 - **Optimistic concurrency via guarded atomic updates** — `approve/reject/ops-approve/ops-reject` all use `findOneAndUpdate` filtered on the *current expected status*, returning a 409 if another actor already changed it, rather than a read-then-write race.
 - **Circuit breaker** — the Zoho client trips after repeated failures.
 - **Request deduplication/caching** — the frontend's axios layer implements its own in-memory de-dup/cache/queue primitives (`RequestDeduplicator` , `PaginationCache` , `ApiCache` , `RequestQueue`) rather than relying on a library like React Query/SWR.
 - **Quote+context text anchoring** — review comments anchor by matching text content (with surrounding context for disambiguation) rather than fragile character offsets, explicitly to survive content edits/reloads.
 - **Folder organization:** feature-agnostic, layer-based (routes/controllers/models on the backend; page/components/hooks/services on the frontend) rather than feature-folder-based (there is no `features/quotations/` grouping, for example) — every quotation-related concern is spread across `controllers/quotationController.js` , `models/quotation.js` , `routes/quotationRoutes.js` , `page/QuotationTemplate.jsx` , `page/ViewQuotationScreen.jsx` , `hooks/useQuotation.js` , etc.
 - **Component hierarchy (quotation authoring, the most complex UI path):** `QuotationScreen` (step orchestration) → `QuotationTemplate` (create-mode form state) or `ViewQuotationScreen` (view/edit-mode) → both render into the shared `QuotationLayout / MobileQuotationLayout` → which render `TermsEditor / TermsViewer` , `ReviewComments` 's `CommentableText / CommentableHtml / CommentBadge` , `PaymentTermPicker` , `TermsTemplateControls` , and the item table/cards.
 - **Data flow:** unidirectional — Zustand stores (or page-local hook state, inconsistently, per hook — see §18) hold data; pages read via hooks; user actions call store actions or direct API-layer functions, which mutate store state on success, which re-renders subscribed components. There is no separate "reducer" layer beyond each Zustand `create()` call's own action functions.
 - **State management:** four independent Zustand stores (`useAppStore` the primary one, plus `useCustomerStore / useItemStore / useNotificationStore`), only a small persisted subset of `useAppStore` (`selectedCompany` , `selectedCurrency` , a few reference-data lists) surviving a page reload via `zustand/persist` .
 - **API layer:** a single `services/api.js` file wraps one configured axios instance with request/response interceptors (auth header injection, company-context header injection, 401 auto-logout, 429 handling) and exposes grouped, resource-oriented API objects (`quotationAPI` , `customerAPI` , etc.) rather than raw axios calls scattered through components.
 - **Database layer:** Mongoose models are the entire "database layer" — no query-builder/repository abstraction beyond what Mongoose itself provides; several models add `.pre('save')` / `.pre(/^find/)` hooks and static/instance helper methods (e.g. `Quotation.getStatsForCompany`) rather than pushing that logic into controllers.
 - **Middleware:** thin, composable Express middleware chain per route (`protect` → `companyContext` → role guard), plus cross-cutting `errorHandler` and `httpLogger` .
 - **Reusable components:** the shared quotation-layout components (`QuotationLayout / MobileQuotationLayout`), the generic `CommentableText / CommentableHtml` review-comment primitives, and the custom listbox-style pickers (`PaymentTermPicker` , `TermsTemplateControls`) are the clearest examples of intentionally shared, parameterized UI in this codebase (built specifically to avoid duplicating the same form across the create and view/edit screens).
-

16. Component Dependency Map

(Representative for the quotation-authoring path — the app's central workflow. Not every page/component/API/model edge in the system is enumerated exhaustively here; this reflects the primary, most-traced flow.)



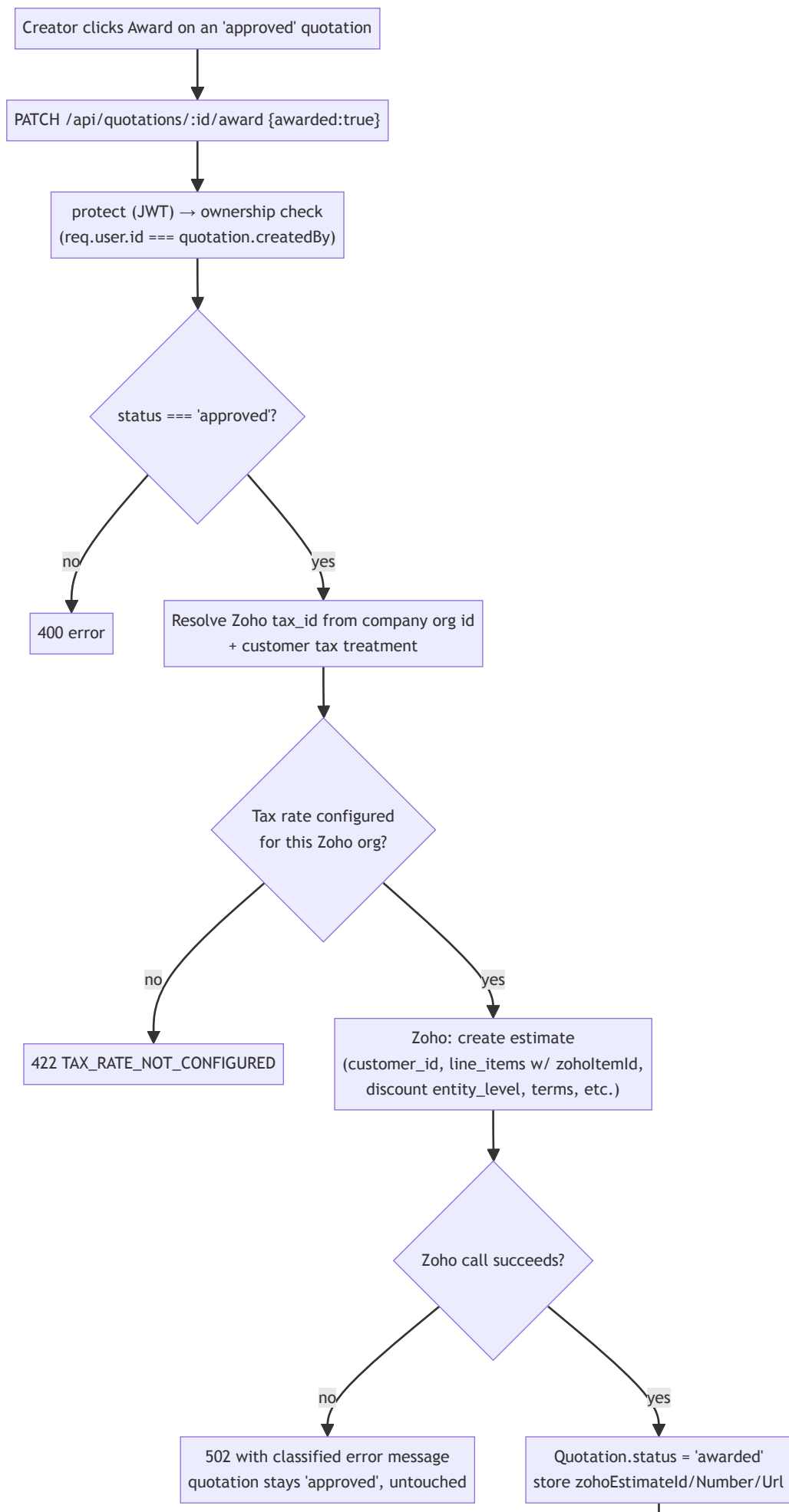
17. End-to-End Data Flow

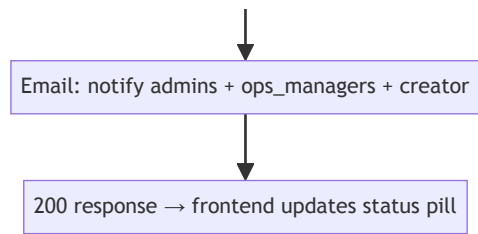
17.1 Example: Creating and submitting a new quotation



Syntax error in text
mermaid version 10.9.6

17.2 Example: Awarding a quotation (Zoho push)





18. Unknown Areas

All items below were confirmed via direct code search (not guessed) unless marked (*inferred*).

18.1 Dead / unwired code

File	Status
backend/controllers/quotationC0ntrollermainfile.js	100% commented out; never required anywhere; an old parallel implementation of company/quotation CRUD (Cloudinary-based, predates the current S3-based controller).
backend/controllers/quotationAwardController.js	A complete, live (not commented out) alternate <code>awardQuotation</code> implementation — never <code>require()</code> 'd anywhere; the real award route points to a same-named function in <code>quotationController.js</code> instead. Missing Saudi 15% VAT support (older tax-ID map).
backend/models/SyncLog.js	Defined, never queried/instantiated anywhere.
backend/utls/approvalWorkflowService.js	Full <code>ApprovalWorkflowService</code> class, never imported. Contains a reference to an undefined <code>Quotation</code> identifier in two of its methods (would throw if ever called) and queries <code>User.find({companyId: ...})</code> even though <code>User</code> has no <code>companyId</code> field.
backend/utls/s3Upload.js	Full S3 helper module, never imported — <code>quotationController.js</code> has its own separately-implemented, same-named functions instead.
backend/utls/itemsHelper.js	Never imported outside itself.
backend/utls/syncLogger.js	Never imported outside itself; its <code>printSummary()</code> method is an empty stub.
backend/utls/logger.js	A second, redundant Winston logger, reachable only from the also-dead <code>cron/exchangeRateJob.js</code> .
backend/cron/exchangeRateJob.js	Fully implemented hourly cron job; never started from <code>server.js</code> (only <code>trnExpiryJob.js</code> is).
backend/migrateQuotations.js (repo root)	Legacy migration script, hardcoded to a local MongoDB URI, not referenced in <code>package.json</code> ; superseded by <code>scripts/migrateQuotationsWithCompany.js</code> .
frontend/.../page/PendingQuotationsScreen.jsx	Entirely commented out.
frontend/.../components/SharedQuotation.jsx	Entirely commented out (1272 lines).
frontend/.../components/Header.jsx , NotificationDrawer.jsx	Defined, not imported/rendered anywhere.
frontend/.../components/index.js	Barrel file importing from three sibling files that don't exist in that folder — broken, and not imported by anything itself.
frontend/.../services/store/apiHandler.js , authSlice.js	An older slice-based store architecture, superseded by the current <code>store.js</code> ; not imported anywhere outside their own folder.

18.2 Incomplete / inconsistent implementations

- **Notification model/API is fully live but has no producer** in the real approval workflow — `utls/notificationService.js` and `utls/approvalWorkflowService.js` (the two modules that *would* write notifications) are not called from `quotationController.js` 's or `adminController.js` 's actual approve/reject/award code paths. The bell/list endpoints work but would return empty data given the current live workflow.
- **duplicateFrom field exists on the Quotation schema but is never persisted** — the frontend sends it when duplicating a quotation, but `createQuotation` 's request-body destructure never reads it, so a "duplicate" quotation has no server-side trace back to its source.
- **quotation.awardedBySnapshot is set in the controller but not declared in the schema** — Mongoose silently drops it (strict mode is on, `strict:false` is not set).
- **Company model has no createdBy / updatedBy fields**, yet `companyController.js` sets them on create/update — also silently dropped.
- **POST /api/exchange-rates/refresh calls a method that doesn't exist** (`ExchangeRateService.forceRefresh`) — would throw a `TypeError` if invoked.
- **Two of NotificationService 's methods (markAsRead / markAllAsRead) reference an undefined identifier** and would throw if called; confirmed never called.
- **Three parallel exchange-rate implementations** exist (`models/quotation.js` 's in-file service, `utls/exchangeRateService.js` , `utls/fetchRates.js`), each with its own hardcoded fallback-rate table and supported-currency list, rather than one shared source.
- **models/items.js exports the Model directly, not as { Item }** , but two consumers (`zoho/customerServices.js` , `zoho/zohoItemsService.js`) destructure it as `{ Item }` — `Item` is undefined in those files' item-DB-write code paths, which would throw if that specific code executed.
- **frontend 's useCustomers / useItemsList / useQuotations hooks are each independently defined 2-3 times** across `customHooks.jsx` / `customerHooks.jsx` / `itemHooks.jsx` with different backing implementations (global store vs. local state vs. direct API calls) — which version a given page uses depends entirely on which file it imports from.
- **frontend/.../hooks/itemHooks.jsx** was not confirmed to be imported by any page examined — `ItemsScreen.jsx` uses `useItemStore` directly instead.
- Three separate PDF-related libraries are installed on the frontend (`jspdf` + `jspdf-autotable` + `html2canvas` , and separately `html2pdf.js`) alongside the actual client-side (`document.write` +`print`) and server-side (Puppeteer) mechanisms — which of the installed libraries is actually exercised at runtime vs. leftover from a prior approach was not fully resolved in this pass.

- `VITE_API_BASE_URL` is defined in the frontend `.env` (`=/api`) but not referenced anywhere in the app code (only `VITE_API_URL` is actually consumed by `services/api.js`) — and `nginx.conf` has no corresponding `/api` proxy rule either, so its intended purpose is unclear.

18.3 Experimental / ambiguous logic

- The in-place "revise a cancelled quotation without creating a new document" branch in `updateQuotation` (`applyRevisionOnSave`) is fully implemented server-side but appears **unreachable from the current frontend** — the live "Edit & Revise" button always routes a cancelled quotation to the create-new-document flow instead. (*Inferred from reading both the button handler and the save logic; not verified via runtime tracing.*)
- `frontend/.../components/UserQuotationStats.jsx` (a 1065-line component) significantly overlaps with `page/UserQuotationStatsPage.jsx` — whether one supersedes the other, or both are used in different contexts, was not resolved in this pass.
- `backend/scripts/importFromExcel.js` is a fully-built historical-data importer not wired to any `package.json` script — its intended operational cadence (one-time backfill vs. recurring) is not stated anywhere in the code. (*Inferred to be a one-time backfill tool from its `--dry-run` flag design, not confirmed.*)
- Whether `backend/Dockerfile`'s exact build steps match the frontend's two-stage pattern was not independently re-verified line-by-line in the final synthesis (a `pm2` ecosystem config's presence was confirmed, implying PM2 is the production process manager, but the Dockerfile's own contents were not re-quoted here).
- Which specific upload flows still route through Cloudinary vs. S3 (both are live-configured) was not fully resolved — the `storageProvider` field's `'cloudinary'` enum value is described in code comments as "legacy," but the Cloudinary upload helper itself is still present and not confirmed dead.

18.4 Not investigated in this pass (scope boundary, not a finding)

- Exact JWT claim shape/expiry duration (confirmed JWT-based via `jsonwebtoken`, but the token payload/TTL was not independently re-quoted from `authController.js` in the final synthesis).
- Full line-by-line contents of `backend/Dockerfile` and `backend/ecosystem.config.js`.
- Full prop-signature-level detail for `ViewQuotationScreen/DocumentManager.jsx`, `QuotationHeaderControls.jsx`, `QuotationMainContent.jsx` (confirmed to exist and be used by `ViewQuotationScreen.jsx`; not traced prop-by-prop).